

Unsloth Cake Demo — Fine-Tuning, Distillation & LLMOps

Sweet Layers Bakery Customer-Support Assistant

Prepared for: Project Submission / Technical Review | Repository: unsloth-cake-demo

Document Purpose

This document provides complete, end-to-end technical documentation of the Fine-Tuning, Distillation & LLMOps project implemented for a bakery customer-support use case. It covers the actual pipeline built, the real evaluation results produced, the challenges encountered during implementation, and the fixes applied — written so that another developer could reproduce the project from this document alone.

1. Project Objective

Sweet Layers Bakery needed a lightweight, on-brand conversational assistant — internally named "Crumb" — capable of answering common customer questions about cakes, ingredients, ordering, delivery, and store policies. The goal of this project was to demonstrate and implement the full lifecycle required to take a small open-source language model from a generic instruction-following base model to a deployable, evaluated, containerized customer-support API.

The project was deliberately scoped around a small, easy-to-inspect task rather than a large production dataset, so that the underlying pipeline — fine-tuning, distillation, evaluation, and serving — remained the primary focus rather than dataset scale. The three stages of the pipeline were built to mirror a realistic LLMOps workflow:

- Fine-Tuning ("bake the custom cake") — adapt a small base model to the bakery's voice and domain using QLoRA.
- Distillation ("bake the cupcake") — transfer the fine-tuned model's behavior into a smaller, cheaper student model.
- LLMOps ("run the bakery") — serve, evaluate, and containerize the resulting model as a production-style API.

This structure was chosen because it reflects how a real team would take a fine-tuned model from a notebook experiment to something a support team could actually rely on and operate.

2. Dataset Preparation

2.1 Dataset Description

The training data is the Sweet Layers Bakery customer-support dataset — a set of realistic customer questions and on-brand assistant replies covering topics such as cake customization, allergies, sizing, pricing, delivery radius, order lead times, and ingredient sourcing. Every example uses a consistent system persona ("Crumb") so the model learns one stable voice rather than several inconsistent tones.

104 training examples were created for this project, each representing one realistic customer support exchange.

2.2 JSONL Chat / Message Format

The dataset is stored as `data/sample_train.jsonl`, with one JSON object per line. Each record uses the standard chat/message format — a list of role-tagged turns (system, user, assistant) — rather than a single flat prompt string. This keeps the data format aligned with how the base model's tokenizer expects conversation to be structured:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are Crumb, the friendly customer support assistant for Sweet Layers Bakery. Answer questions warmly, concisely, and accurately about our cakes, ingredients, ordering, delivery, and policies. Keep replies to 2-4 sentences."
    },
    {
      "role": "user",
      "content": "Hi, I need help with a cake order."
    },
    {
      "role": "assistant",
      "content": "Absolutely! I'd be happy to help with your cake order. Just tell me what you'd like, and we'll take it from there!"
    }
  ]
}
```

Using the chat/message format — rather than a hand-rolled prompt string — matters because the base model, tokenizer, and inference server must all agree on exactly how a conversation is serialized into tokens. Formatting the data this way lets the tokenizer's own chat template (see Section 3) be applied consistently at training, evaluation, and inference time, avoiding a common and easy-to-miss source of silent fine-tuning bugs.

2.3 Held-Out Evaluation Prompts

A separate file, `data/eval_prompts.jsonl`, holds 5 held-out customer questions that never appear in the training set, each paired with a reference answer. These prompts (e.g. "Can you make a cake for someone with a severe egg allergy?", "How early do I need to book a three-tier wedding cake?") were used consistently across every evaluation step in the project — base vs. fine-tuned comparison, teacher vs. student comparison, and the automated CI-style eval gate — so results are comparable across stages.

3. Fine-Tuning

3.1 Base Model

The base model used is `unsloth/Llama-3.2-1B-Instruct-bnb-4bit`, a 1-billion-parameter instruction-tuned Llama 3.2 model pre-quantized to 4-bit by Unsloth. A small model was chosen deliberately so the entire pipeline — including training — could run on a single consumer-class or free-tier GPU rather than requiring a multi-GPU cluster.

3.2 QLoRA and LoRA

Full fine-tuning updates every parameter in a model, which is expensive in both compute and memory. This project instead used QLoRA (Quantized Low-Rank Adaptation):

- **4-bit quantization:** the base model's weights are loaded in 4-bit precision, drastically cutting the GPU memory required just to hold the model.

- **LoRA adapters:** instead of updating the full weight matrices, small trainable low-rank "adapter" matrices are injected alongside the frozen base weights. Only these adapter matrices — a few megabytes — are trained and saved.

Together, QLoRA made it possible to fine-tune the model with a small memory footprint while still producing a meaningfully specialized model, and it kept the saved artifact small and portable (an adapter, not a full model copy).

3.3 LoRA Configuration

Parameter	Value	Purpose
LoRA rank (r)	16	Capacity vs. cost trade-off for the adapter matrices.
LoRA alpha	16	Scaling factor applied to the adapter's contribution.
LoRA dropout	0.0	Disabled — Unsloth's training path is optimized for dropout = 0.
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	Attention and MLP projection layers adapted by LoRA.
Gradient checkpointing	"unsloth"	Unsloth's memory-efficient checkpointing implementation.

3.4 Training Configuration

Hyperparameter	Value
Epochs	3
Per-device batch size	2
Gradient accumulation steps	4
Learning rate	2e-4
Warmup steps	5
Weight decay	0.01
LR scheduler	linear
Max sequence length	1024
Seed	3407

3.5 Chat Template Formatting

Training text was never hand-assembled. Every example was formatted by calling the tokenizer's own `tokenizer.apply_chat_template()` method on the raw messages list, which guarantees the exact same special tokens, role markers, and turn structure the base model was originally trained to expect. The same call is reused at inference time in the FastAPI service and during evaluation, so the model never sees an unfamiliar prompt shape at any stage of the pipeline.

3.6 Adapter Output

Training was run with a Hugging Face TRL SFTTrainer wrapping the Unsloth-loaded model. After training completed, the LoRA adapter and tokenizer were saved to `fine_tuning/outputs/lora-adapter/` — a small directory (a few megabytes) rather than a full model checkpoint, since only the adapter weights differ from the public base model.

4. Fine-Tuning Evaluation

4.1 Comparison Method

`fine_tuning/inference_compare.py` loads both the original base model and the fine-tuned (LoRA-merged) model, runs both on the same 5 held-out prompts from `data/eval_prompts.jsonl`, and scores every generated answer against its reference answer using ROUGE-L. The base and fine-tuned answers are printed side by side so quality can be checked qualitatively, not just numerically — a higher ROUGE-L score does not always mean a genuinely better customer-facing answer.

On the held-out set, the fine-tuned model generally produced shorter, more on-brand, more directly responsive answers than the base model, which tended to over-explain and drift from the bakery's voice.

4.2 Automated Eval Gate Results

A stricter, CI-style automated gate (`llmops/eval.py`) re-ran the fine-tuned model against the same held-out prompts and enforced a minimum-quality and maximum-latency threshold, exiting with a non-zero status on failure so it can block a deployment pipeline. The actual results from this run were:

Metric	Result	Threshold / Limit	Outcome
Average ROUGE-L	0.224	≥ 0.20	PASSED
Average latency	2.24 seconds	≤ 15 seconds	PASSED

Evaluation Result: PASSED

Average ROUGE-L (0.224) cleared the 0.20 quality threshold, and average response latency (2.24s) was well within the 15-second limit. The eval gate exited with status 0, meaning this model version would be allowed to proceed to deployment in a CI/CD pipeline.

What this means in practice: ROUGE-L measures overlap between the generated answer and a human-written reference answer, so a score of 0.224 indicates the fine-tuned model's phrasing and content meaningfully overlap with the expected on-brand response, without being an exact copy (which would be neither expected nor desirable for natural-sounding replies). The latency figure confirms the model is fast enough for an interactive support-chat use case, well under the 15-second ceiling considered acceptable for this project.

5. Knowledge Distillation

5.1 Purpose

Distillation was performed to explore whether a smaller, cheaper-to-run model could reproduce most of the fine-tuned model's behavior, trading a small amount of quality for meaningfully faster inference — a realistic trade-off for a high-traffic customer-support endpoint where latency and serving cost matter.

5.2 Teacher and Student

- **Teacher:** the fine-tuned QLoRA model from Section 3 (fine_tuning/outputs/lora-adapter), used frozen — no gradients — purely to generate soft-label outputs for the student to learn from.
- **Student:** a smaller student model trained through knowledge distillation, using a lower-rank LoRA adapter ($r = 8$, versus the teacher's $r = 16$) and a shorter max sequence length (512 vs. 1024), so it is cheaper to run at inference time.

5.3 Teacher → Student Knowledge Transfer

The student was trained with a combined distillation loss that blends two signals: how closely the student's output distribution matches the teacher's softened output distribution (KL-divergence on soft labels), and how well the student predicts the actual training tokens (standard cross-entropy):

```
loss = ALPHA * KL(student_soft || teacher_soft) + (1 - ALPHA) * cross_entropy(student, true_tokens)
```

```
Temperature = 2.0 # softens both distributions before the KL term
Alpha       = 0.7 # weight on imitating the teacher vs. matching ground truth
Epochs     = 2
Learning rate = 1e-4
```

A higher temperature spreads probability mass across more candidate tokens, exposing the teacher's "runner-up" choices — the extra signal that makes soft-label distillation more informative than training on hard text labels alone. Alpha = 0.7 was chosen to lean toward imitating the teacher's style while still anchoring the student to the real training text.

5.4 Student Evaluation and Comparison

distillation/evaluate_student.py re-ran both the teacher and the student on the same 5 held-out prompts and recorded parameter counts, ROUGE-L quality, and latency for each. The actual measured results were:

Metric	Teacher (fine-tuned)	Student (distilled)
Total parameters	760,547,328	750,979,072
Average ROUGE-L	0.213	0.241
Average latency	2.861 s	1.021 s

Distillation Result

The student was roughly 2.8x faster than the teacher on average latency, with a small parameter-count reduction (~1.3%). On this held-out set the student's average ROUGE-L (0.241) was actually slightly higher than the teacher's (0.213) — a quality gap of about -0.028, meaning the student did not lose quality on this small evaluation sample while responding substantially faster.

Because this project intentionally kept the student on the same small base architecture for portability, the parameter-count reduction is modest — most of the size/cost saving here comes from the smaller LoRA rank rather than a different underlying architecture. Swapping in a genuinely smaller base model for the student would be the natural next step to obtain a larger size and speed advantage.

5.5 Generated Student Artifacts

The trained student adapter and tokenizer were saved to distillation/outputs/student-model/, and the full comparison — summary statistics plus every per-prompt teacher/student answer pair and score — was written to distillation/outputs/student_vs_teacher.json for later inspection or reporting.

6. LLMOps / Model Serving

6.1 FastAPI Service

The trained model is exposed through a thin FastAPI application (llmops/app.py). The API layer intentionally contains only routing, request validation, and response shaping — the model itself is loaded lazily on first request so the process starts quickly and /health responds immediately even before the (slow) model load finishes. All configuration — model path, system prompt, max tokens, 4-bit loading — is read from environment variables rather than hardcoded, so the same container image can point at different model versions in different environments.

6.2 /health Endpoint

A GET /health endpoint returns a simple liveness check, {"status": "ok", "model_path": ...}, used by Docker's health check and by any orchestrator or load balancer to confirm the service is up before routing traffic to it.

6.3 /generate Endpoint

A POST /generate endpoint accepts a JSON body of the form {"prompt": "..."} (with an optional max_new_tokens override), validates that the prompt is non-empty, and returns the generated reply along with the measured latency and the model path that produced it:

```
POST /generate
{"prompt": "Do you have any gluten-free cakes?"}

Response:
{
  "response": "...",
  "latency_seconds": 2.1,
  "model_path": "/app/fine_tuning/outputs/lora-adapter"
}
```

6.4 Request / Response Flow

On each generate request, the incoming prompt is wrapped with the bakery's system persona and passed through the tokenizer's chat template (the same call used during training and evaluation), tokenized, and passed to the model's generate() call. Latency is measured around the generation call itself, and the decoded text — with special tokens stripped — is returned to the caller alongside that latency figure.

6.5 Loading the Model for Inference

At request time, the service loads the base model together with the trained LoRA adapter through Unsloth's FastLanguageModel.from_pretrained(), pointed at the adapter directory (MODEL_PATH), then switches the model into inference mode. This mirrors exactly how the model was loaded during evaluation and distillation, so behavior is consistent across every stage of the pipeline.

7. Dockerization

7.1 Dockerfile

The API was containerized using a single-stage Dockerfile based on python:3.11-slim. Build-essential system packages are installed first (required by torch / bitsandbytes at runtime), Python dependencies are installed before application code is copied in (so dependency layers are cached across code-only changes), and both the trained model artifacts and the application code are copied into the image. A Docker HEALTHCHECK calls the /health endpoint every 30 seconds to confirm the container is serving correctly.

7.2 Build Command

The image is built from the repository root — not the llmops/ folder — so the build context can also include the trained adapter under fine_tuning/outputs/:

```
docker build -f llmops/Dockerfile -t cake-bakery-api .
```

7.3 Run Command and Port Mapping

The container exposes port 8000 internally, mapped to the host's port 8000:

```
docker run -p 8000:8000 cake-bakery-api

# Optionally override the model path or generation limits at runtime:
docker run -p 8000:8000 \
-e MODEL_PATH=/app/fine_tuning/outputs/lora-adapter \
-e MAX_NEW_TOKENS=150 \
cake-bakery-api
```

7.4 Container Health Check Result

Container Verified

The containerized API's health check successfully returned {"status":"ok"}, confirming the FastAPI service started correctly inside Docker and the model path resolved as expected.

8. Problems / Challenges Faced

Challenge	Impact	Resolution
Local Windows environment was CPU-only, and the available local GPU had only 2 GB of VRAM.	Unsloth's GPU detection and 4-bit (bitsandbytes) quantization path could not run reliably locally; QLoRA fine-tuning and distillation were not practical on this hardware.	Training and distillation were moved to Google Colab, running on a Tesla T4 GPU, which provided enough VRAM to load the 4-bit base model and train LoRA adapters at a reasonable speed.
bitsandbytes 4-bit dependency issue during API inference.	The serving API's model-loading step depended on the same 4-bit quantization stack used in training, which is sensitive to the exact platform/CUDA combination it runs on.	The environment used for serving was aligned with a Linux/CUDA-compatible setup (matching the training environment's dependency versions) so the same 4-bit loading path used during training and evaluation also worked at inference time.
Docker dependency installation was large and slow.	The PyTorch + Unsloth + bitsandbytes stack pulled in large binary wheels, making the Docker image large and the pip install step slow, particularly on the first build.	The Dockerfile installs dependencies before copying application code so this expensive layer is cached across code-only rebuilds; the first build remains slow but subsequent builds reuse

Challenge	Impact	Resolution
		the cached dependency layer and are much faster.
Docker had initial dependency / model-loading issues.	Early container builds failed or errored when the model artifacts or dependency versions inside the image did not match what the serving code expected.	The build was restructured to copy requirements.txt and llmops/requirements.txt separately from application code, and to explicitly copy the trained fine_tuning/outputs/ directory into the image, ensuring the container always has a matching model artifact and dependency set.

9. Final Architecture / Pipeline

The complete pipeline, in the order it actually runs, is:

```

Dataset (data/sample_train.jsonl, 104 examples)
  |
  v
QLoRA Fine-Tuning (fine_tuning/train_lora.py)
  |
  v
LoRA Adapter / Teacher (fine_tuning/outputs/lora-adapter)
  |
  v
Knowledge Distillation (distillation/distill.py)
  |
  v
Smaller Student Model (distillation/outputs/student-model)
  |
  v
Evaluation (fine_tuning/inference_compare.py, distillation/evaluate_student.py,
            llmops/eval.py -- automated CI-style gate)
  |
  v
FastAPI Serving (llmops/app.py -- /health, /generate)
  |
  v
Docker Container (llmops/Dockerfile -- cake-bakery-api image)

```

Each stage's output is the next stage's input: the fine-tuned adapter becomes the distillation teacher, and the same held-out prompt set is reused for every evaluation step so results stay comparable end to end.

Repository Structure

```

unsloth-cake-demo/
├── data/
│   ├── sample_train.jsonl    # 104 training examples (chat format)
│   └── eval_prompts.jsonl    # 5 held-out evaluation prompts

```

```

├── fine_tuning/                                # ACT 1 -- QLoRA fine-tuning
│   ├── config.py
│   ├── train_lora.py
│   ├── inference_compare.py
│   ├── outputs/
│   │   └── lora-adapter/
│   │       └── before_after_comparison.json
│   └── README.md
├── distillation/                             # ACT 2 -- teacher -> student KD
│   ├── config.py
│   ├── distill.py
│   ├── evaluate_student.py
│   ├── outputs/
│   │   └── student-model/
│   │       └── student_vs_teacher.json
│   └── README.md
├── llmops/                                  # ACT 3 -- serve, evaluate, containerize
│   ├── app.py
│   ├── eval.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── outputs/eval_gate_result.json
│   └── README.md
├── requirements.txt
└── README.md

```

10. Final Results

The following were successfully completed and verified:

- A 104-example bakery customer-support training dataset, in chat/message JSONL format, plus a held-out 5-prompt evaluation set.
- A fine-tuned LoRA adapter (QLoRA, $r = 16$) on unsloth/Llama-3.2-1B-Instruct-bnb-4bit, saved and reusable.
- A distilled, smaller student model trained via teacher $\rightarrow$ student knowledge distillation.
- A repeatable evaluation pipeline covering base-vs-fine-tuned comparison, teacher-vs-student comparison, and an automated ROUGE-L/latency gate.
- An automated eval gate result of ROUGE-L = 0.224 and average latency = 2.24 seconds — PASSED against the 0.20 / 15-second thresholds.
- A FastAPI serving layer exposing /health and /generate, with configuration driven entirely by environment variables.
- A successfully built and run Docker container (cake-bakery-api) exposing the API on port 8000, with a verified `{"status": "ok"}` health check.

Deliverable	Status
Training dataset (104 examples)	Complete

Deliverable	Status
QLoRA fine-tuned adapter	Complete
Distilled student model	Complete
Evaluation pipeline	Complete
Automated eval gate (ROUGE-L 0.224, 2.24s)	PASSED
FastAPI serving API	Complete
Dockerized API	Complete — health check verified

11. Reflection / What We Would Do Differently

11.1 Start Directly on a Suitable GPU Environment

Discovering the local Windows/2 GB VRAM limitation only after attempting training cost setup time. Starting the project with a Colab (or other cloud) T4 GPU session from the outset — rather than first attempting local training and pivoting afterward — would have avoided the wasted iteration on an environment that was never going to support 4-bit training.

11.2 A Lighter Serving Image

The current Docker image installs the same requirements.txt used for training and distillation, including the full Unsloth/PyTorch/bitsandbytes stack, which makes the serving image large and slow to build. A production-oriented next step would be a serving-only dependency set — or a merged, non-4-bit inference model — so the deployed image only carries what /generate actually needs at runtime.

11.3 Separate Training and Inference Dependencies

Related to the above, keeping one shared requirements.txt for both training and serving couples two workloads that have different needs: training needs trl, peft, and the full quantization/training stack, while serving only needs enough to load a model and generate text. Splitting these more cleanly (rather than layering llmops/requirements.txt on top of the same root file) would shrink the deployable image and reduce the chance of a training-only dependency breaking inference.

11.4 Expand Evaluation Beyond ROUGE-L on 5 Prompts

The current evaluation gate is intentionally small — 5 held-out prompts scored only with ROUGE-L — which is enough to demonstrate the pipeline but not enough to fully trust in production. Expanding the held-out set, adding complementary metrics (e.g. semantic similarity, factuality/faithfulness checks against bakery policy facts, or a lightweight LLM-as-judge pass), and tracking results over time would give a much more reliable signal before shipping a model change.

End of document.